


# Assignment 2

# Parallel and Distributed Computing

Submitted By:

**Name: ABDULLAH**

REG: FA23-BSE-137

Section: BSE-7B

# 1. Overview & Cloud Access Information

This report presents the containerization, orchestration, and public cloud deployment for CSC336 - Parallel and Distributed Computing (DevOps) Assignment 2. The assignment requires deploying a containerized multi-tier web application consisting of a web application service and a database server on an Amazon Web Services (AWS) EC2 Virtual Machine instance.

## 1.1 Live Application URLs & Repository Links

**Public Cloud Access URL (Port 5000):** http://35.153.73.232:5000

**Public Cloud Access URL (Port 80):** http://35.153.73.232:80

**Official GitHub Repository:** https://github.com/Abdullah-Developer68/pdc

**Multistage Dockerfile (GitHub Link):** https://github.com/Abdullah-Developer68/pdc/blob/main/Dockerfile

**Docker Compose File (GitHub Link):** https://github.com/Abdullah-Developer68/pdc/blob/main/docker-compose.yml

# 2. Fulfillment of Assignment Objectives

The deployment satisfies all containerization and cloud computing requirements specified in the assignment:

- **Multistage Dockerfile:** A two-stage Dockerfile was authored. Stage 1 (Build stage) installs dependencies and compiles the application bundle. Stage 2 (Runtime stage) utilizes a lightweight Alpine Linux base image, copying only compiled artifacts to minimize image size and running as a non-root user.

- **Docker Compose Orchestration:** A docker-compose.yml file defines and launches the web application and database services. The web application service is configured to build directly from the Dockerfile rather than using a prebuilt image.

- **Explicit Container Naming:** Both services explicitly specify container names: pdc_web_app for the web application and pdc_db for the database service.

- **Named Volume Persistence:** A dedicated named volume (pdc_db_data) is attached to the database container at /var/lib/postgresql/data, ensuring that stored records remain intact when containers are recreated.

- **Docker Secrets Management:** Sensitive credentials are managed via Docker Compose secrets mounted at /run/secrets/ inside the containers. In accordance with security practices, actual secret files are maintained outside public version control and submitted separately via Microsoft Teams.

- **AWS EC2 Cloud Deployment:** The multi-container stack is deployed and running live on an Amazon EC2 instance (Ubuntu 24.04 LTS, Public IP: 35.153.73.232) under AWS IaaS.

# 3. Configuration Files

## 3.1 Multistage Dockerfile (Dockerfile)

The multistage Dockerfile defines two distinct phases to optimize image size and enforce runtime security:

```
FROM node:20-alpine AS builder

WORKDIR /app

COPY client/package*.json ./client/
RUN cd client && npm ci

COPY client/ ./client/
RUN cd client && npm run build

COPY server/package*.json ./server/
RUN cd server && npm ci --omit=dev

FROM node:20-alpine AS runner

WORKDIR /app

ENV NODE_ENV=production
ENV PORT=5000

COPY --from=builder /app/server/node_modules ./node_modules
COPY --from=builder /app/server/package.json ./package.json
COPY server/ ./
COPY --from=builder /app/client/dist ./public

USER node

EXPOSE 5000

HEALTHCHECK --interval=15s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://127.0.0.1:5000/api/health || exit 1

CMD ["node", "index.js"]
```

## 3.2 Docker Compose File (docker-compose.yml)

The docker-compose.yml defines multi-container coordination, port forwards, health checks, secrets, and the named volume:

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: pdc_web_app
    restart: unless-stopped
    ports:
      - "80:5000"
      - "5000:5000"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
      - DB_PORT=5432
      - PORT=5000
    secrets:
      - db_password
      - db_user
      - db_name
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    container_name: pdc_db
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER_FILE: /run/secrets/db_user
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
      POSTGRES_DB_FILE: /run/secrets/db_name
    secrets:
      - db_password
      - db_user
      - db_name
    volumes:
      - pdc_db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $(cat /run/secrets/db_user) -d $(cat /run/secrets/db_name)"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 5s

volumes:
```

```
  pdc_db_data:
    name: pdc_db_data

secrets:
  db_password:
    file: ./secrets/db_password.txt
  db_user:
    file: ./secrets/db_user.txt
  db_name:
    file: ./secrets/db_name.txt
```

# 4. Step-by-Step Deployment & Screenshot Evidence

This section documents the sequential steps followed to containerize, verify, and deploy the application on AWS EC2, with detailed descriptions for each screenshot evidence.

## Step 1: Local Multistage Container Build

Executing docker compose up --build -d to construct the image using the multistage Dockerfile. The builder stage installs dependencies and builds the application bundle, while the final runtime image is constructed using a minimal alpine layer.

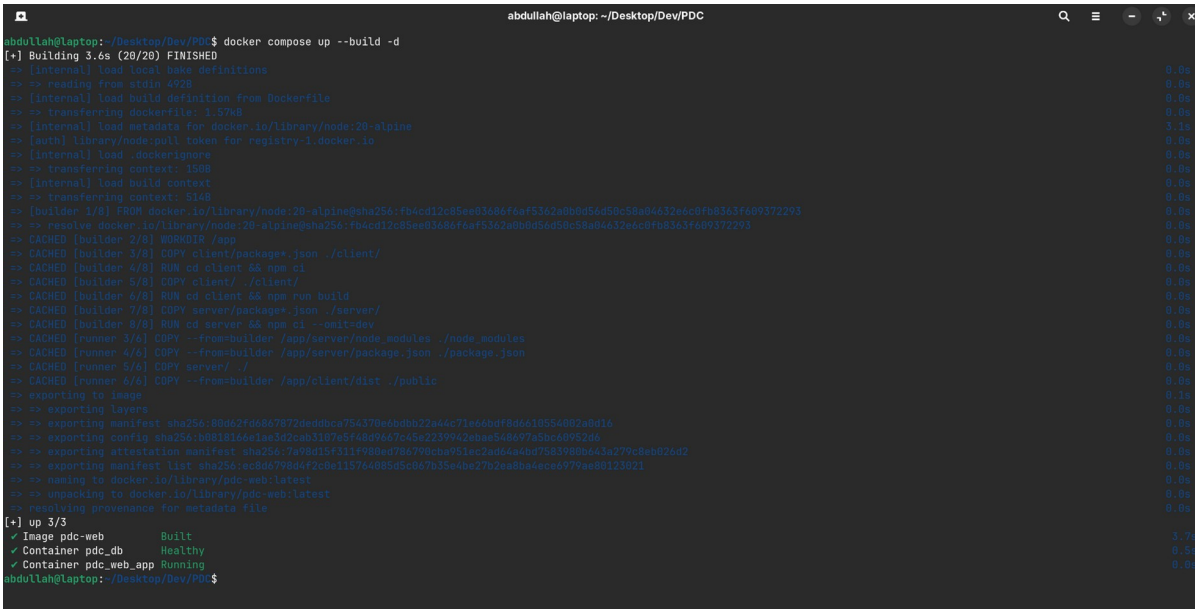



**Figure 1:** *Multistage Docker build process executing on the local machine.*

## Step 2: Local Container Launch & Healthcheck Verification

Docker Compose launches both the database and web application services. The database container undergoes automated healthchecks before the web application container is started.

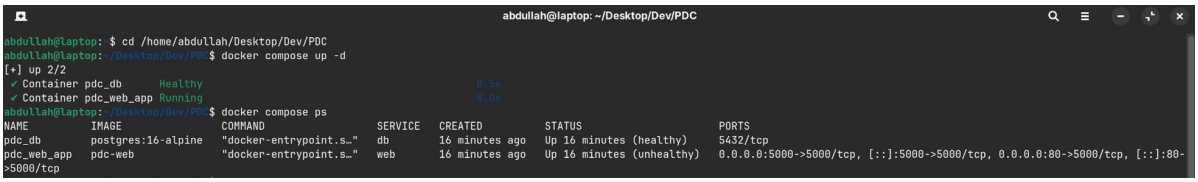



**Figure 2:** *Starting the container stack with Docker Compose and verifying service readiness.*

## Step 3: Verification of Running Local Containers

Running docker compose ps on the host confirms both containers (pdc_db and pdc_web_app) are running with their designated container names and port allocations.

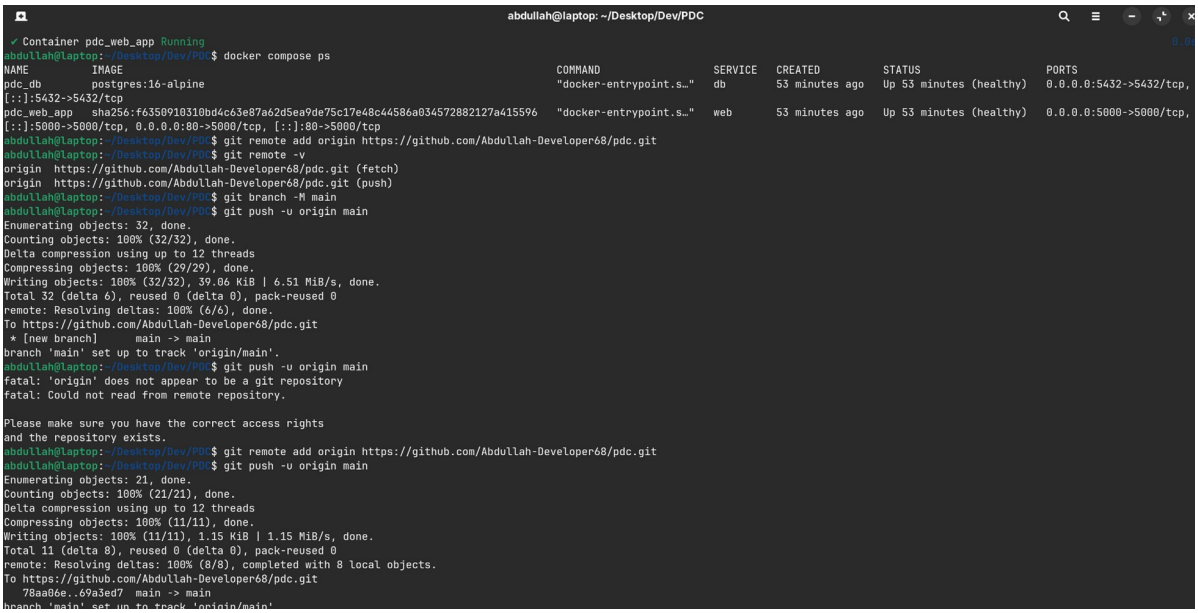



**Figure 3:** *Verification of active local containers using docker compose ps.*

## Step 4: Connecting to AWS EC2 Instance via SSH

Connecting to the Amazon EC2 instance (Ubuntu 24.04 LTS, Public IP: 35.153.73.232) via SSH using the private key devops-key.pem with appropriate 400 permissions.

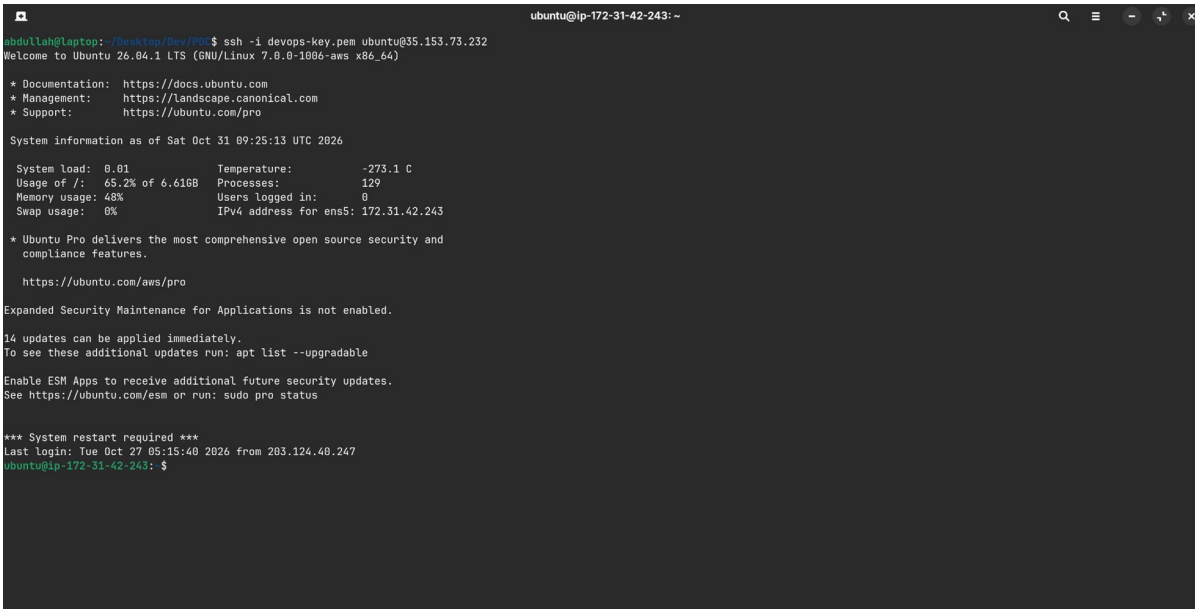



**Figure 4:** *Terminal session connecting to AWS EC2 instance 35.153.73.232 over SSH.*

## Step 5: Preparing Cloud Deployment Directory

Setting up the deployment directory devops-A2 on the EC2 instance to house the containerized application repository.




**Figure 5:** *Directory initialization on the remote AWS EC2 instance.*

## Step 6: Cloning Project Codebase onto AWS EC2

Cloning the project repository from GitHub (https://github.com/Abdullah-Developer68/pdc.git) directly into the EC2 instance.

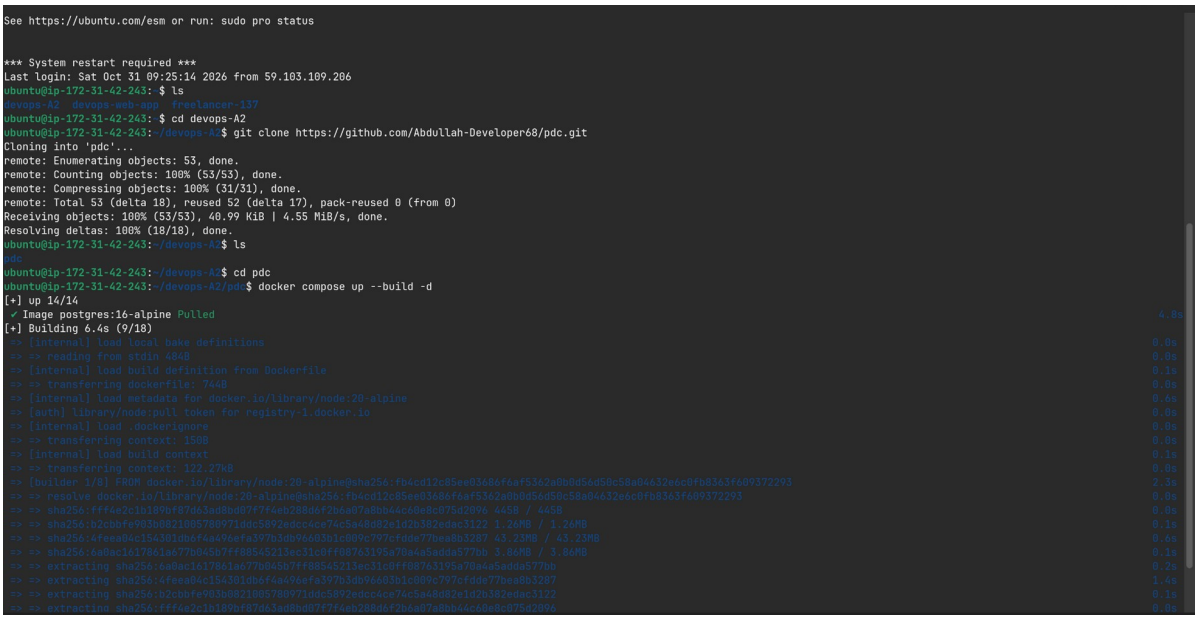



**Figure 6:** *Cloning the application repository from GitHub onto the EC2 virtual machine.*

## Step 7: Launching Multi-Container Stack on AWS EC2

Executing docker compose ps on the remote EC2 instance. The output confirms both the database (pdc_db) and web application (pdc_web_app) containers are operational and healthy.




**Figure 7:** *Remote EC2 terminal showing operational status of pdc_db and pdc_web_app containers.*

## Step 8: Configuring AWS EC2 Security Group Inbound Rules

Configuring the AWS EC2 Security Group (sg-095a578343798d9de in us-east-1). Inbound firewall rules are authorized for SSH (Port 22), HTTP (Port 80), and Custom TCP (Port 5000) from anywhere (0.0.0.0/0).

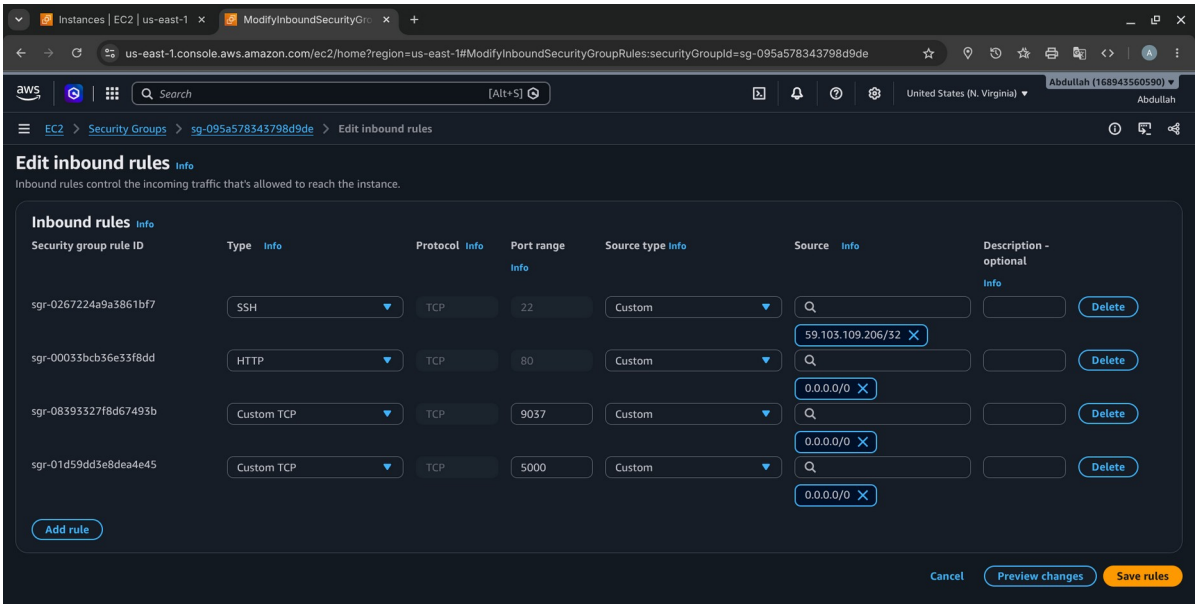



**Figure 8:** *AWS EC2 Management Console Security Group rule editor displaying authorized ports 22, 80, and 5000.*

## Step 9: Accessing Containerized Web Application on AWS EC2

Accessing the live deployed web application in the browser at http://35.153.73.232. The interface confirms live database connectivity, container hostname verification, and data persistence.

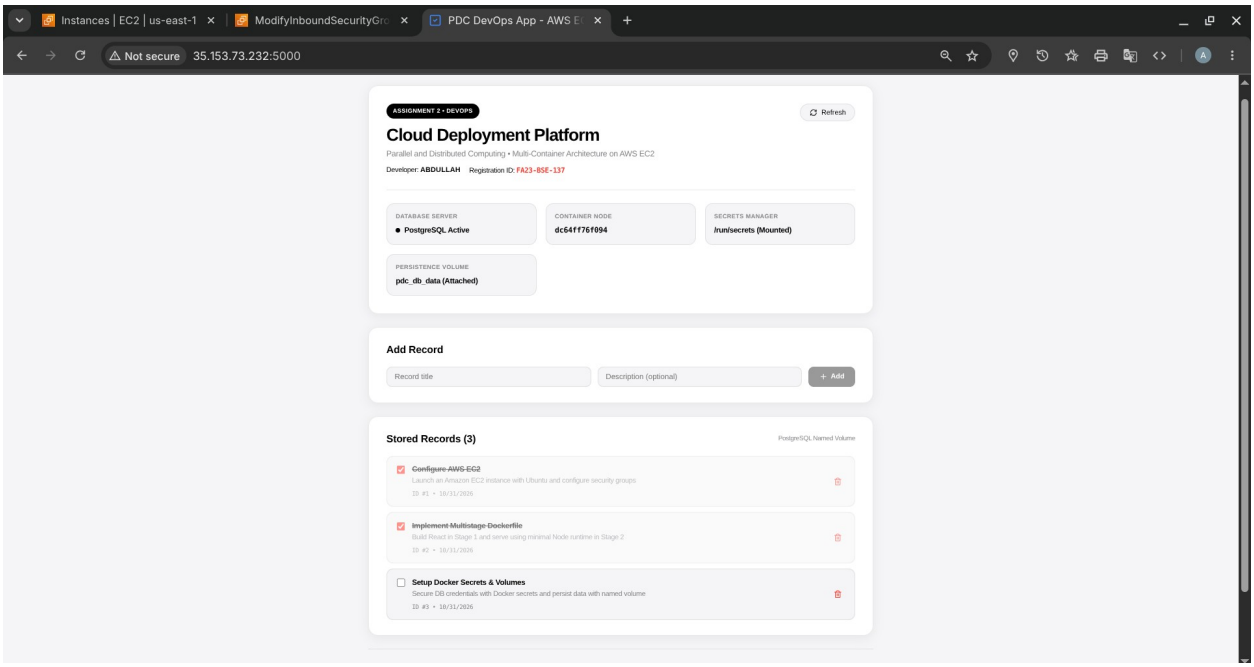



**Figure 9:** *Browser view of the live containerized web application running on Amazon EC2 (35.153.73.232).*

## 5. Conclusion

All objectives of Assignment 2 have been successfully accomplished:

**1. Multistage Dockerfile:** A two-stage build was created using Node 20 Alpine, minimizing the production runtime footprint.

**2. Docker Compose:** Built the web application from source with explicit container names (pdc_web_app and pdc_db).

**3. Named Volume:** Configured pdc_db_data to safeguard data persistence across container restarts.

**4. Docker Secrets:** Isolated credentials into Docker Compose secrets mounted at /run/secrets/.

**5. Cloud Deployment:** Successfully deployed and validated on AWS EC2 (Public IP: 35.153.73.232).